(Don't) Forget About It: Forgetting-Penalized Supervised Learning*

Gaurav Sood†

March 22, 2026

Abstract

During continued training, a machine-learning model can begin misclassifying examples it previously labeled correctly—a within-task instability often called model regression. While catastrophic forgetting across different tasks has been studied for decades, standard supervised objectives still optimize only average accuracy and ignore these per-example reversals, even though each flip can trigger costly downstream work in production pipelines that rely on stable behavior. We compare six methods for preventing model regression: three penalty-based approaches (confidence drop, fixed anchor, selective distillation), two constraint-based approaches (projected gradient descent, backwards compatible weight interpolation), and a standard baseline. In comprehensive experiments across five benchmark datasets with 10 random splits each, we find that constraint-based methods vastly outperform penalty-based methods. Projected gradient descent with Negative Flip Rate (NFR) constraints achieves the best trade-off, winning on 3 of 5 datasets by exploring the full feasible region through iterative projection. Post-hoc weight interpolation (BCWI) is competitive and simpler, winning on 2 datasets. Penalty-based methods provide some benefit over baseline but plateau at higher NFR values. These findings establish that training-time NFR constraints are more effective than penalty-based regularization for preventing model regression.

1 Introduction

Machine-learning models deployed in production are routinely retrained or fine-tuned as new data arrive. Although these updates generally *improve* overall accuracy, they can sometimes cause the same model to misclassify examples it previously labeled correctly. We refer to

*https://github.com/finite-sample/pareto-gd.

†Gaurav can be reached at

each such reversal as *model regression*. This phenomenon resembles catastrophic forgetting (French, 1999) but occurs *within a single supervised task*, rather than across clearly separated tasks in continual-learning benchmarks. While catastrophic forgetting has inspired a rich body of work in multi-task settings (Kirkpatrick et al., 2017; Zenke et al., 2017), the practical consequences of model regression in production pipelines have received far less attention.

Model regression is more than a statistical curiosity: in human-in-the-loop systems it incurs operational costs. When performance drops for a specific subgroup, organizations must reallocate domain specialists, launch retraining programs, or absorb higher error-handling expenses. Section 2 quantifies these costs through a staffing-optimization case study and shows that even modest regressions can erase the financial gains of an accuracy update.

Standard supervised learning objectives minimize average loss and provide no incentive to preserve correctness on *known-good* examples. Consequently, training dynamics may sacrifice stability for small aggregate improvements, exposing production systems to disruptive swings in behavior.

In this paper, we compare six methods for preventing model regression:

1. **Baseline**—standard ERM with no regression prevention.
2. **Confidence Drop Penalty**—penalizes any per-example loss increase vs the previous epoch.
3. **Fixed Anchor Penalty**—penalizes when candidate loss exceeds incumbent loss on an anchor set.
4. **Selective Distillation**—distills to incumbent predictions on an anchor set.
5. **Projected Gradient Descent**—trains with ERM, then projects to NFR $\leq \varepsilon$ after each epoch.
6. **Backwards Compatible Weight Interpolation (BCWI)**—post-hoc interpolation between incumbent and candidate weights.

Our contributions are threefold:

1. We formalize *model regression* and provide an economic framework that links per-example reversals to real operational costs.
2. We systematically compare six methods spanning penalty-based, constraint-based, and post-hoc approaches.
3. In comprehensive experiments across five datasets with 10 splits each, we demonstrate that constraint-based methods (projected GD, BCWI) substantially outperform penalty-based methods.

2 Operational Cost Case Study: Customer-Operations Staffing

To ground the economic stakes of model regression, consider a financial services firm that uses an ML classifier to triage incoming customer requests and detect cases requiring human review. Requests are routed into categories—*account inquiries*, *fraud alerts*, *loan applications*, *technical support*, and so on. Each category has a dedicated team of specialists who intervene when the model's confidence is low or when it commits an error.

2.1 Economic Drivers

- **Specialization efficiency**: Employees work fastest and most accurately within their domain expertise (e.g., fraud analysts versus loan officers).
- **Retraining cost**: Moving workers between domains requires expensive retraining that temporarily reduces productivity.
- **Hiring and separation cost**: Adjusting headcount entails recruiting, onboarding, or severance expenses.
- **Accuracy benefit**: Higher model accuracy lowers the total volume of human interventions, reducing labor demand across all categories.

2.2 The Regression Cost Problem

When updating the ML model from version M_A to M_B, the company faces a trade-off. While M_B may achieve higher overall accuracy than M_A, it may perform worse on specific subgroups or categories. This creates two types of costs:

Staffing Realignment Costs: If model M_B has worse performance on fraud detection but better performance on loan processing, staff must be moved from loan processing to fraud detection roles, incurring retraining costs.

Accuracy Benefits: If model M_B has sufficiently higher overall accuracy, fewer total staff are needed, creating cost savings.

Let $C_{move}(M_A, M_B)$ denote the cost of staff realignment when transitioning from M_A to M_B, and let $C_{staff}(M)$ denote the ongoing staffing cost for model M. The company should only deploy M_B if:

$$C_{staff}(M_A) - C_{staff}(M_B) > C_{move}(M_A, M_B) \tag{1}$$

2.3 Forgetting-Penalized Training as Economic Optimization

This business case provides clear economic justification for forgetting-penalized training. Rather than accepting any model with higher overall accuracy, the company can use penalties to constrain model updates such that:

1. **Subgroup Performance Preservation**: Penalize decreases in performance on individual categories to minimize staff reallocation needs
2. **Minimum Improvement Threshold**: Only accept model updates if the overall accuracy improvement exceeds a threshold τ that justifies operational disruption
3. **Pareto-Aware Updates**: Prefer model improvements that help some categories without harming others

The penalty-based objective becomes:

$$\mathcal{L}_{economic}(\theta) = \mathcal{L}_{std}(\theta) + \lambda_{econ} \sum_{k=1}^{K} \max(0, \ell_k(M_A) - \ell_k(M_\theta)) \tag{2}$$

where $\ell_k(M)$ represents the loss of model M on category k, and the penalty term discourages performance decreases on any category.

2.4 Broader Implications

This operational perspective suggests that "regression" in machine learning should be understood not merely as a technical phenomenon, but as an economic externality. The cost of operational disruption from model updates may outweigh the benefits of marginal accuracy improvements, particularly in:

- **Human-in-the-loop systems** where model changes affect workforce allocation
- **Multi-stakeholder environments** where different groups depend on model performance for specific use cases
- **Regulated industries** where model changes require compliance reviews and approval processes
- **Customer-facing applications** where inconsistent model behavior affects user experience and trust

This economic framing provides a principled foundation for determining when forgetting prevention is worthwhile and how to set appropriate penalty weights λ based on operational costs rather than purely technical considerations.

3 Related Work

3.1 Example Forgetting as a Dataset Diagnostic

Toneva et al. (2019) introduced the notion of a *forgetting event*—a correct $\rightarrow$ incorrect transition for a single example during SGD—and showed that (i) forgetting counts follow a heavy-tailed distribution, (ii) the ranking of (un)forgettable examples is architecture-invariant, and

(iii) pruning the *never-forgotten* fraction of CIFAR-10 does not harm test accuracy. Follow-up work leverages forgetting scores for data curation (Paul et al., 2021) and long-tailed re-weighting (Yu and Chen, 2022). In contrast, we embed a *differentiable* forgetting cost directly into the loss to prevent harmful flips in the first place.

3.2 Catastrophic forgetting and continual learning

Catastrophic forgetting—loss of performance on earlier tasks while learning new ones—was first noted by French (1999). Continual-learning research has since produced three main families of defenses:

Parameter-level regularizers. Elastic Weight Consolidation (EWC) (Kirkpatrick et al., 2017), Memory-Aware Synapses (MAS) (Zenke et al., 2017), and Synaptic Intelligence (SI) (Zenke et al., 2017) estimate parameter importance and penalize updates on important weights. EWC, for example, uses the diagonal Fisher information,

$$F_i \;=\; \mathbb{E}_{(x,y)\sim\mathcal{D}_{\text{old}}}\Big[\big(\tfrac{\partial}{\partial\theta_i}\log p(y \mid x,\theta)\big)^2\Big],$$

and adds a quadratic penalty $\frac{\lambda}{2}\sum_i F_i(\theta_i - \theta_i^\star)^2$ to the new task loss. These methods are effective when task boundaries are known, but require storing (or recomputing) importance estimates and do not directly address within-task regression.

Rehearsal buffers. Methods such as Gradient Episodic Memory (Lopez-Paz and Ranzato, 2017) replay a subset of past examples while learning new ones, thereby preserving previous performance at the cost of extra memory and computation.

Architectural expansion. Progressive networks (Rusu et al., 2016) and related approaches allocate new capacity for each task, sidestepping interference but sacrificing parameter efficiency.

Our work differs in scope: we remain within *one* supervised task and operate at the *output level*, tracking only one bit per example and requiring no task boundaries or replay.

3.3 Multi-objective and Pareto-optimal training

Multi-task learning can be cast as a multi-objective problem in which each task loss is an independent objective. Lin et al. (2019) solve a set of constrained sub-problems in parallel to approximate the Pareto front, producing a spectrum of accuracy–fairness (or other) trade-offs that users can select post-training. Navon et al. (2021) go further, training a hypernetwork that maps a preference vector to weights, thus generating the entire Pareto front in a single run.

These methods aim to *offer choices* along a frontier of trade-offs. In contrast, our penalties pursue a single, conservative update: they bias SGD toward changes that improve—or at least do not worsen—every example's loss. The goal is operational stability,

not exploration of a full trade-off surface, making our approach better suited to production pipelines that value consistency on known-good cases.

3.4 Summary

Prior work either analyzes example-level forgetting after training, protects entire tasks via parameter or rehearsal methods, or explores Pareto trade-offs across tasks.

Our contribution is orthogonal: a lightweight, example-level regularizer that prevents within-task regression *during* training and aligns directly with operational cost constraints in real-world deployments.

4 Methods

4.1 Problem Formulation

Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ be a training dataset, where $x_i \in \mathbb{R}^d$ and $y_i \in \{0, 1\}$ for binary classification. We denote the model parameters as θ and the prediction function as $f_\theta(x)$. The standard training objective minimizes the empirical risk:

$$\mathcal{L}_{std}(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell(f_\theta(x_i), y_i) \tag{3}$$

where ℓ is the loss function (e.g., binary cross-entropy).

As motivated by the operational cost analysis in Section 2, we seek to augment this objective with penalty terms that discourage regression on previously learned examples, particularly when the operational benefits of accuracy improvements do not justify the costs of performance changes on specific subgroups or categories.

4.2 Classification Flip Penalty

Our first approach explicitly penalizes forgetting events—instances where an example transitions from being correctly classified to incorrectly classified between consecutive epochs.

Let $\theta^{(t)}$ denote the model parameters at epoch t, and define the correctness indicator:

$$c_i^{(t)} = \mathbb{I}[\text{sign}(f_{\theta^{(t)}}(x_i) - 0.5) = y_i] \tag{4}$$

The forgetting penalty at epoch t is:

$$\mathcal{R}_{flip}^{(t)}(\theta) = \sum_{i=1}^{n} \mathbb{I}[c_i^{(t-1)} = 1 \text{ and } c_i^{(t)} = 0] \tag{5}$$

The total training objective becomes:

$$\mathcal{L}_{flip}^{(t)}(\theta) = \mathcal{L}_{std}(\theta) + \lambda_{flip} \cdot \mathcal{R}_{flip}^{(t)}(\theta) \tag{6}$$

where $\lambda_{flip} \geq 0$ controls the strength of the forgetting penalty.

4.3 Confidence Drop Penalty

The Confidence Drop approach implements a more general "do no harm" principle by penalizing any increase in per-example loss, not just classification flips.

For each example i, we track the per-example loss from the previous epoch:

$$\ell_i^{(t-1)} = \ell(f_{\theta^{(t-1)}}(x_i), y_i) \tag{7}$$

The Confidence Drop penalty encourages the current loss to not exceed the previous loss:

$$\mathcal{R}_{conf}^{(t)}(\theta) = \sum_{i=1}^{n} \max(0, \ell(f_\theta(x_i), y_i) - \ell_i^{(t-1)}) \tag{8}$$

The complete objective is:

$$\mathcal{L}_{conf}^{(t)}(\theta) = \mathcal{L}_{std}(\theta) + \lambda_{conf} \cdot \mathcal{R}_{conf}^{(t)}(\theta) \tag{9}$$

4.4 Fixed Anchor Penalty

The Fixed Anchor approach uses the incumbent model's loss as a fixed target on an anchor set. For each example in the anchor set where the incumbent was correct, we penalize when the candidate's loss exceeds the incumbent's loss:

$$\mathcal{R}_{anchor}(\theta) = \sum_{i \in \mathcal{A}} \max(0, \ell(f_\theta(x_i), y_i) - \ell(f_{\theta_{inc}}(x_i), y_i)) \tag{10}$$

where $\mathcal{A}$ is the anchor set (examples where the incumbent was correct) and θ_{inc} are the incumbent model parameters.

4.5 Selective Distillation

Selective Distillation distills the candidate model to match the incumbent's predictions on examples where the incumbent was correct. Using soft targets from the incumbent:

$$\mathcal{R}_{distill}(\theta) = \sum_{i \in \mathcal{A}} \mathrm{KL}(p_{inc}(x_i) \| p_\theta(x_i)) \tag{11}$$

where $p_{inc}(x_i) = f_{\theta_{inc}}(x_i)$ is the incumbent's predicted probability.

4.6 Projected Gradient Descent

Projected GD takes a fundamentally different approach: instead of adding penalties, it enforces the NFR constraint directly. The algorithm initializes from the incumbent and, after each gradient step, projects back to the feasible region where NFR $\leq \varepsilon$:

Input: Incumbent θ_{inc}, target NFR ε, validation set
Initialize $\theta^{(0)} \leftarrow \theta_{inc}$;
for *epoch* $t = 1, \ldots, T$ **do**
  $\theta' \leftarrow \theta^{(t-1)} - \alpha \nabla_\theta \mathcal{L}_{std}(\theta^{(t-1)})$;
  Compute NFR(θ') on validation set;
  if $NFR(\theta') > \varepsilon$ **then**
    Find β^* via binary search such that NFR($\beta^* \theta_{inc} + (1 - \beta^*)\theta'$) $= \varepsilon$;
    $\theta^{(t)} \leftarrow \beta^* \theta_{inc} + (1 - \beta^*)\theta'$;
  end
  else
    $\theta^{(t)} \leftarrow \theta'$;
  end
end

Algorithm 1: Projected Gradient Descent for NFR Constraint

This allows the model to explore the full feasible region through iterative projection.

4.7 Backwards Compatible Weight Interpolation (BCWI)

BCWI is a post-hoc approach that does not modify training. It first trains a candidate model via standard ERM, then finds an interpolation weight α such that the interpolated model achieves the target NFR:

$$\theta_{bcwi} = \alpha \cdot \theta_{inc} + (1 - \alpha) \cdot \theta_{cand} \tag{12}$$

Binary search finds the smallest α (closest to the candidate) such that NFR(θ_{bcwi}) $\leq \varepsilon$. BCWI is simpler than projected GD but restricted to the 1D line segment between incumbent and candidate.

4.8 Implementation Details

Penalty methods are applied after a warmup period to allow the model to stabilize before enforcing regression constraints. Algorithm 2 presents the general training procedure for penalty-based methods.

Input: Dataset $\mathcal{D}$, penalty type, λ, warmup epochs T_{warmup}
Initialize model parameters $\theta^{(0)}$;
$t \leftarrow 0$;
while *not converged* **do**
  Compute standard loss: $\mathcal{L}_{std}(\theta^{(t)})$;
  if $t > T_{warmup}$ **then**
    Compute penalty: $\mathcal{R}^{(t)}(\theta^{(t)})$;
    Total loss: $\mathcal{L}_{total} = \mathcal{L}_{std} + \lambda \cdot \mathcal{R}^{(t)}$;
  end
  else
    Total loss: $\mathcal{L}_{total} = \mathcal{L}_{std}$;
  end
  Update parameters: $\theta^{(t+1)} \leftarrow \theta^{(t)} - \alpha \nabla_\theta \mathcal{L}_{total}$;
  $t \leftarrow t + 1$;
end

Algorithm 2: Penalty-Based Training with Regression Prevention

5 Experiments

5.1 Datasets

We evaluate all methods on five binary classification datasets from OpenML:

Table 1. Dataset Statistics

Dataset	Train	Test	Features	Task
Adult	32,561	16,281	105	Income prediction
Bank	31,647	13,564	51	Marketing response
Credit	21,000	9,000	23	Default prediction
Diabetes	537	231	8	Health outcome
Spambase	3,220	1,381	57	Spam detection

We use a multi-layer perceptron (MLP) with two hidden layers of 64 units each, ReLU activations, and a sigmoid output layer. All models are trained using the Adam optimizer with a learning rate of 0.001.

5.2 Experimental Protocol

For each dataset, we run 10 random train/validation/test splits. Each split:

1. Trains an *incumbent* model on the "old" subset (1/3 of training data)
2. Trains a *candidate* model on the full training data using each method

3. Evaluates on a held-out test set

For penalty-based methods (confidence drop, fixed anchor, selective distillation), we sweep penalty strength $\lambda \in \{0.0, 0.5, 1.0, 2.0, 4.0, 8.0\}$ with 10 warmup epochs. For constraint-based methods (projected GD, BCWI), we sweep NFR targets $\varepsilon \in \{0.0, 0.005, 0.01, 0.015, 0.02, 0.$

5.3 Evaluation Metrics

We measure:

- **Negative Flip Rate (NFR)**: Fraction of examples where incumbent was correct but candidate is incorrect
- **Accuracy**: Classification accuracy on held-out test set

Methods are ranked by the lowest NFR achieved while maintaining accuracy within 1% of the baseline.

5.4 Results

Table 2 shows method rankings across all datasets. Projected GD and BCWI consistently outperform penalty-based methods.

Table 2. Method Rankings Across Datasets (lower rank = better)

Dataset	1st	2nd	3rd	4th	5th	6th
Adult	bcwi	proj_gd	fixed_anc	sel_dist	conf_drop	baseline
Bank	proj_gd	bcwi	conf_drop	fixed_anc	sel_dist	baseline
Credit	bcwi	proj_gd	fixed_anc	sel_dist	conf_drop	baseline
Diabetes	proj_gd	bcwi	sel_dist	fixed_anc	conf_drop	baseline
Spambase	proj_gd	bcwi	fixed_anc	conf_drop	sel_dist	baseline

Table 3. Average Rank and Win Count

Method	Avg Rank	#1 Wins
projected_gd	1.40	3
bcwi	1.60	2
fixed_anchor	3.40	0
selective_distill	4.20	0
confidence_drop	4.40	0
baseline	6.00	0

Table 4 shows the best NFR achieved by each method while maintaining accuracy within 1% of baseline.

Table 4. Best NFR Achieved (maintaining ≥99% baseline accuracy)

Dataset	proj_gd	bcwi	fixed_anc	sel_dist	conf_drop	baseline
Adult	0.0003	0.0000	0.0303	0.0309	0.0329	0.0413
Bank	0.0001	0.0002	0.0270	0.0276	0.0259	0.0317
Credit	0.0013	0.0008	0.0161	0.0163	0.0209	0.0235
Diabetes	0.0288	0.0347	0.0555	0.0520	0.0663	0.0732
Spambase	0.0073	0.0099	0.0145	0.0160	0.0153	0.0179

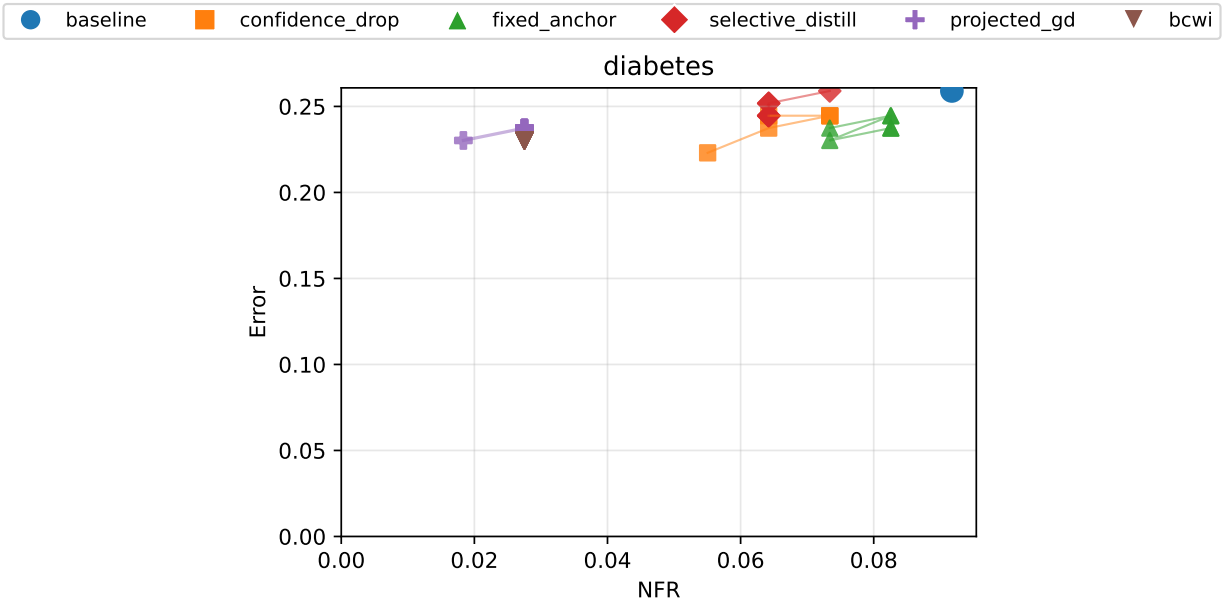

Figure 1. Pareto frontiers (Error Rate vs NFR) for all methods across five datasets. Constraint-based methods (projected_gd, bcwi) consistently achieve lower NFR at matched accuracy.

Figure 1 shows the Pareto frontiers across all datasets.
Key observations:

- Constraint-based methods (projected GD, BCWI) achieve NFR < 0.01 on most datasets while maintaining accuracy
- Penalty-based methods plateau at NFR ∼0.015–0.03, providing limited improvement over baseline
- Projected GD wins on 3/5 datasets by exploring the full feasible region via iterative projection
- BCWI is competitive (wins 2/5) and simpler, requiring no modifications to training
- All methods improve over baseline, which always ranks last

6 Discussion

6.1 Why Projected GD Outperforms BCWI

Projected GD wins on 3/5 datasets while BCWI wins on 2/5. The key difference lies in the feasible regions they explore:

- **BCWI** is restricted to the 1D line segment between incumbent weights θ_{inc} and candidate weights θ_{cand}. The optimal point on this line may not achieve the best accuracy-NFR trade-off.
- **Projected GD** explores the full feasible region through iterative projection. Each epoch takes a gradient step toward lower loss, then projects back to NFR $\leq \varepsilon$ if needed. This allows the model to find paths through weight space that BCWI cannot reach.

BCWI wins on Adult and Credit, suggesting these datasets have simpler feasible regions where the incumbent-candidate line is already near-optimal. Projected GD wins on Bank, Diabetes, and Spambase, where iterative exploration finds better solutions.

6.2 Why Penalty Methods Plateau

Penalty-based methods add a regularization term to the loss but do not directly constrain NFR. This creates two problems:

1. **Indirect control**: The penalty strength λ does not map directly to a target NFR. Higher λ reduces NFR but also reduces accuracy, often more than necessary.
2. **Epoch-local comparisons**: Confidence drop compares to the previous epoch, not the incumbent. This can drift from the incumbent over many epochs.

Fixed anchor and selective distillation compare to the incumbent but still use indirect penalty-based control.

6.3 When to Use Each Method

- **Projected GD**: Best for new model training when the NFR constraint is critical. Requires knowing the target NFR upfront.
- **BCWI**: Simpler, good for retrofitting existing models. No retraining needed—just interpolate post-hoc.
- **Penalty methods**: Useful when the NFR target is unknown a priori, but expect limited NFR reduction.

6.4 Limitations

1. **Interpolation assumption**: Both projected GD and BCWI assume NFR is monotonic in the interpolation weight, which holds empirically but is not guaranteed.

2. **Binary classification**: All experiments use binary classification; extension to multi-class requires defining NFR appropriately.

3. **Architecture scope**: We use MLPs; results may differ for other architectures (CNNs, transformers).

7 Conclusion

We have systematically compared six methods for preventing model regression: three penalty-based approaches (confidence drop, fixed anchor, selective distillation), two constraint-based approaches (projected gradient descent, backwards compatible weight interpolation), and a standard baseline.

Our comprehensive evaluation across five datasets with 10 random splits each yields clear conclusions:

1. **Constraint-based methods vastly outperform penalty-based methods.** Projected GD and BCWI consistently achieve NFR < 0.01 while maintaining accuracy, whereas penalty methods plateau at NFR $\sim$0.015–0.03.

2. **Training-time constraints beat post-hoc interpolation** on 3/5 datasets. Projected GD's iterative projection explores the full feasible region, finding better trade-offs than BCWI's 1D line search.

3. **BCWI is competitive and simpler.** It wins on 2/5 datasets and requires no modification to training—just post-hoc interpolation.

4. **All methods improve over baseline.** The baseline always ranks last, confirming that model regression is a real problem worth addressing.

For practitioners, we recommend projected GD when NFR constraints are critical and the target is known upfront, and BCWI when simplicity and retrofitting existing models are priorities. Penalty-based methods remain useful when the NFR target is unknown, but practitioners should expect limited NFR reduction.

This work establishes that treating NFR as a hard constraint during training is more effective than penalty-based regularization. As ML systems become increasingly embedded in production pipelines where backwards compatibility matters, these findings provide practical guidance for preventing model regression.

References

French, R. M. (1999). Catastrophic forgetting in connectionist networks. *Trends in cognitive sciences*, 3(4):128–135.

Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., et al. (2017). Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114(13):3521–3526.

Lin, X., Zhen, H.-L., Li, Z., Zhang, Q.-F., and Kwong, S. (2019). Pareto multi-task learning. *Advances in Neural Information Processing Systems*, 32.

Lopez-Paz, D. and Ranzato, M. (2017). Gradient episodic memory for continual learning. *Advances in neural information processing systems*, 30.

Navon, A., Shamsian, A., Chechik, G., and Fetaya, E. (2021). Learning the pareto front with hypernetworks. *International Conference on Learning Representations*.

Paul, M., Ganguli, S., and Dziugaite, G. K. (2021). Deep learning on a data diet: Finding important examples early in training. *Advances in neural information processing systems*, 34:20596–20607.

Rusu, A. A., Rabinowitz, N. C., Desjardins, G., Soyer, H., Kirkpatrick, J., Kavukcuoglu, K., Pascanu, R., and Hadsell, R. (2016). Progressive neural networks. *arXiv preprint arXiv:1606.04671*.

Toneva, M., Sordoni, A., Tachet des Combes, R., Trischler, A., Bengio, Y., and Gordon, G. J. (2019). An empirical study of example forgetting during deep neural network learning. In *International Conference on Learning Representations (ICLR)*.

Yu, H. and Chen, J. (2022). Class-balanced loss based on example forgetting for long-tailed recognition.

Zenke, F., Poole, B., and Ganguli, S. (2017). Continual learning through synaptic intelligence. *International conference on machine learning*, pages 3987–3995.